Examen Final - Estructuras de Datos y Algoritmos (2025-1)

Preg Inorder-stack traversal (6 puntos)

En esta implementación se debe utilizar el código base: preg-inorder.cpp
Se debe desarrollar la función **StackInOrder** para recorrer el árbol en orden, pero sin usar la recursividad del lenguaje c++ y usando una estructura de datos Stack.
La función main no debe ser modificada.

Preg Patterns (7 puntos)

En esta implementación se debe utilizar el código base: preg-patterns.cpp. En la función main hay una matriz de caracteres. Esta matriz contiene varios patrones como este:

```
##
#
```

En esta pregunta, se pide implementar la función **searchPattern** para contar la cantidad de patrones que aparecen en la matriz. Este proceso se debe realizar con un número de threads entre 4 a 8.

```
char matrix[10][10] = {
    {'.', '#', '#', '.', '.', '.', '#', '#', '.', '.'},
    {'.', '.', '.', '.', '.', '.', '#', '.', '#', '#'},
    {'.', '.', '#', '#', '.', '.', '.', '.', '#', '.'},
    {'.', '.', '#', '.', '.', '.', '.', '.', '.', '.'},
    {'#', '#', '.', '.', '#', '#', '.', '#', '#', '.'},
    {'.', '.', '.', '.', '#', '.', '.', '.', '.', '.'},
    {'.', '.', '.', '.', '.', '#', '#', '.', '.', '.'},
    {'.', '.', '.', '#', '#', '#', '.', '#', '#', '.'},
    {'.', '#', '#', '#', '.', '.', '.', '#', '.', '.'},
    {'.', '#', '.', '.', '.', '.', '.', '.', '#', '#'}
};
```

Al final del main se debe mostrar las coordenadas de inicio de los patrones encontrados en este formato:
(9,1)(2,2)(3,1)

Preg MaxHeap (7 puntos)

En esta pregunta se debe utilizar el código base: preg-maxheap.cpp. En la función main de este programa hay tres arrays, que son estructuras max-heaps ya armadas. Se pide implementar la función XMayores , que debe mostrar los x valores mayores del heap. En el programa hay dos funciones de apoyo: HijoDer() e HijoIzq(), que pueden ayudar en el proceso.
En el proceso, no se debe ordenar el array, y no se debe recorrer el array completo. Se puede utilizar alguna estructura adicional.

```
MaxHeap: [95, 83, 90, 75, 80, 89, 70, 60, 65, 55, 45, 87, 40, 35, 25]
Índices: [ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14]

                        *95*
                      /       \
                    83         *90*
                  /    \      /    \
                75      80   *89*   70
               / \     / \   /  \   / \
             60  65   55 45 *87*40 35 25
```

```
MaxHeap: [100, 95, 90, 80, 85, 88, 75, 70, 78, 82, 83, 86, 73, 72, 71]
Índices: [ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9,  10, 11, 12, 13, 14]

                        *100*
                      /       \
                    *95*       *90*
                  /    \      /    \
                80      85   *88*  75
               / \     / \   / \   / \
             70  78   82 83 86 73 72 71
```

```
MaxHeap: [98, 95, 92, 91, 85, 80, 75, 70, 88, 82, 83, 78, 76, 73, 72]
Índices: [ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14]

                        *98*
                      /       \
                    *95*       *92*
                  /    \      /    \
               *91*    85    80    75
               / \     / \   / \   / \
             70  88   82 83 78 76 73 72
```